

Convolutional Recurrent Neural Networks: Implementation for Volatility Prediction with Trading Times Analysis

Denys Shkola*, Markus Gruber†, Orest Mykhailiuk‡

Abstract—This paper investigates the use of CRNN model, based on combined convolutional and recurrent neural layers, for predicting the future volatility of financial assets. The chosen three model is implemented via Keras Tensorflow framework. Using S&P 500 (SPX) and CBOE Volatility Index (VIX) datasets, the data is preprocessed for time-series forecasting and model performance is evaluated based on custom analysis model for best trading times.

Index Terms—CRNN, Volatility Prediction, Q-learning, Real-Time Trading, Payday Anomaly

I. INTRODUCTION

Financial time series are highly noisy, nonlinear, and exhibit both short- and long-range dependencies. Capturing such dynamics is essential for forecasting volatility and making informed trading decisions. While traditional models like ARIMA and GARCH fail to model nonlinear temporal patterns effectively, deep learning architectures offer powerful alternatives.

Among these, Recurrent Neural Networks (RNNs) such as LSTMs and GRUs have shown promise in capturing sequential dependencies. However, they often struggle to extract localized patterns and are prone to vanishing gradients over long sequences. On the other end, attention-based models like Transformers are computationally expensive and require large datasets for stable convergence conditions that are often impractical in financial domains with limited labeled data.

To address these limitations, Convolutional Recurrent Neural Network (CRNN) architecture is adopted. CRNNs combine 1D convolutional layers for local pattern recognition with recurrent layers for temporal sequence modeling. This hybrid approach enables efficient learning of both short-term structures (e.g., technical indicators) and long-term market trends, making CRNNs a scalable and more data-efficient choice for volatility prediction.

In this work, a CRNN-based forecasting model integrated with reinforcement learning is developed and deployed to identify optimal trading times. Its performance is validated on SPX, VIX, and AAPL data streams, and its application in real-time trading is demonstrated via broker APIs.

II. LITERATURE REVIEW

A. Payday Anomaly and Market Behavior

The payday anomaly refers to statistically significant patterns around the start of each month where indices like SPX exhibit higher average returns with reduced downside volatility

[1]. These short-term behaviors have inspired strategies using leveraged instruments, though limitations like data sparsity require robust models. Deep learning, specifically sequence models, provides a natural fit for capturing such temporal dependencies [2].

B. Benefits of CRNN in Financial Forecasting

Convolutional Recurrent Neural Networks (CRNNs) combine the feature extraction power of CNNs with the temporal sequence modeling of RNNs. Unlike standalone LSTMs/GRUs that often miss short-term patterns or CNNs that lack long-term context, CRNNs provide a balanced mechanism, extracting both temporal and local structures [3]. In trading, this hybrid model has been shown to outperform basic RNNs or CNNs in both speed and accuracy, while being more computationally efficient than attention-based models like Transformers [2], [4].

C. Bayesian and Reinforcement-Based Trading Models

Bayesian approaches allow uncertainty modeling, providing predictive distributions rather than point forecasts. This is particularly useful in financial markets where uncertainty quantification (e.g., via Monte Carlo Dropout) supports risk-aware decisions [5]. Reinforcement Learning, particularly Q-learning and Deep Q Networks (DQNs), enables trading agents to learn optimal policies through interaction with market environments. These agents maximize cumulative returns by adapting to feedback and have been applied to dynamic portfolio management and trade execution [6], [7]. While more complex, such models offer adaptive decision-making in volatile markets.

III. METHODOLOGY

A. Data Collection

Financial data was collected using the Alpaca API for the SPX and VIX indices. Additionally, AAPL data was used for microstructure-level trading time analysis. These datasets provide both market performance (SPX) and sentiment-driven volatility signals (VIX). Due to data limitations for VIX before 1990, the time range was restricted accordingly. Retrieved features included: Open, High, Low, Close, Volume, VWAP, and Trade Count. Real-time data was later streamed via WebSocket connections for live inference.

B. Data Preprocessing

The datasets were merged on timestamps and cleaned by forward-filling missing values. Technical indicators were engineered using both classic (e.g., Moving Average, RSI, EMA) and statistical transformations (e.g., log returns, lagged returns). Features were normalized using `MinMaxScaler` to aid in neural network convergence.

The final dataset was split chronologically into training, validation, and test sets.

C. Model Architecture

The proposed CRNN model integrates convolutional and recurrent components to process multi-dimensional time series. The architecture consists of:

- 1D Convolutional layers for detecting local patterns in technical indicators.
- ReLU activation for non-linearity.
- GRU or LSTM recurrent layers to capture temporal dependencies.
- Dropout layers for regularization.
- Fully connected Dense layers for mapping to regression output.

The model was implemented in TensorFlow/Keras and allows modular selection between GRU, LSTM, and vanilla RNNs via parameter injection. Loss was measured using Mean Squared Error (MSE), with Mean Absolute Error (MAE) and directional accuracy (MDA) as evaluation metrics.

D. Training and Hyperparameter Optimization

Model training followed a mini-batch gradient descent approach with early stopping. Hyperparameters such as the number of recurrent units, learning rate, dropout, and batch size were tuned using Optuna's Bayesian optimization strategy. Optimal configurations were chosen based on validation MAE and generalization performance.

To address the challenge of choosing the best time to trade, a Q-learning agent was introduced. This agent was trained using historical predictions, reward signals (based on directional accuracy and profit), and feedback from simulated trades. A discretized time-slot environment was constructed to identify when model confidence aligned with historically profitable action zones.

E. Prediction and Market Application

The final CRNN model was used to forecast near-future volatility and direction. Output predictions were converted to position signals through thresholding and signal aggregation. Based on Q-agent recommendations, trades were simulated using a strategy combining predictive returns with VIX-based insurance weights (via linear regression).

To test real-world viability, the full pipeline was deployed with live data ingestion through WebSocket feeds, executing trades via broker APIs (Alpaca and Lynx). This closed-loop system allowed automated signal generation, risk-adjusted trade sizing, and conditional order placement—all in real-time.

REFERENCES

- [1] Quantpedia, "Payday anomaly," 2024, <https://quantpedia.com/strategies/payday-anomaly/>.
- [2] Y. Zhu, N. Laptev, and B. Zong, "Deepvol: Volatility forecasting from high-frequency data with dilated causal convolutions," *arXiv preprint arXiv:2004.13408*, 2020.
- [3] N. Kumar and P. Singh, "Hybrid cnn-rnn models for financial time-series forecasting: A comparative review," *Journal of Financial Data Science*, vol. 4, no. 1, pp. 15–30, 2022.
- [4] S.-H. Tsang, "Convolutional rnn: An enhanced model for extracting features from sequential data," 2020, <https://sh-tsang.medium.com/brief-review-rcnn-recurrent-convolutional-neural-network-for-object-recognition-8c9870>.
- [5] C. Blundell, J. Cornebise, K. Kavukcuoglu, and D. Wierstra, "Weight uncertainty in neural networks," *arXiv preprint arXiv:1505.05424*, 2015.
- [6] J. Moody and M. Saffell, "Learning to trade via direct reinforcement," in *Proceedings of the 12th International Conference on Neural Information Processing Systems*, 2001, pp. 1049–1055.
- [7] Y. Deng, F. Bao, Y. Kong, Z. Ren, and Q. Dai, "Deeprader: A reinforcement learning approach for automated stock trading," *Proceedings of IJCAI*, 2016.